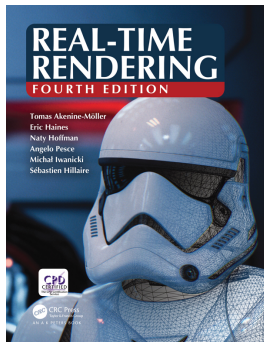


Rasterization Pipeline

Introduction to Computer Graphics - Lecture 03

Pirmin Pfeifer



Real-Time Rendering,
Fourth Edition
T. Akenine-Möller Et Al.

[View Online](#)

Related Chapters:

- ▶ 2 The Graphics Pipeline
- ▶ 3 The Graphics Processing Unit

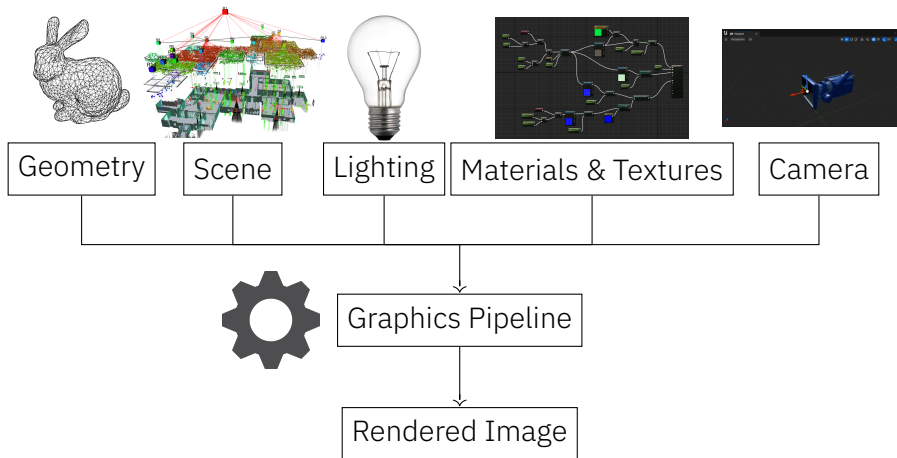
A deep dive on the topic can be found in the book ["A trip through the Graphics Pipeline"](#).

Pipeline

- ▶ Algorithm that consist out of multiple sub task (stages)
- ▶ Data is handed from step to step
- ▶ Stages depend linearly on each other (per item/data)
- ▶ Stages can typically be executed in parallel

When a pipeline is executed in parallel, the slowest stage determines the throughput of the pipeline and is called the bottleneck.

The Graphics Pipeline¹



¹Graph not exhaustive.

The pipeline is specialized to process vertices and triangle

Vertex

A point in 3D space

Triangle

Three ordered vertices define a triangle together

Mesh

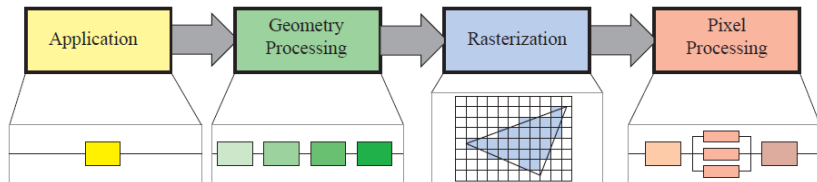
A collection of (mostly connected) triangles

Shader

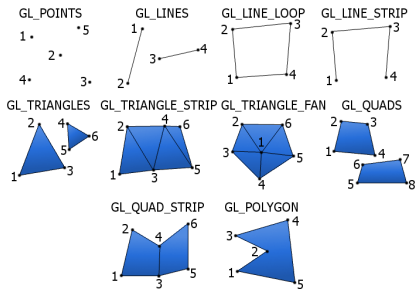
A program that runs on the GPU. It is commonly used in the Graphics pipeline to:

- ▶ Transform and calculate vertex-positions
- ▶ Determine pixel-colors
- ▶ Do arbitrary calculations

The Graphics Pipeline - Stages



- ▶ Dispatch *rendering primitives* into the pipeline (Draw Calls)
- ▶ Configure later pipeline stages
- ▶ Done mostly on CPU



OpenGL Rendering Primitives



The Geometry Processing Stage operates on a per vertex basis.

Main Tasks:

- ▶ Transform vertex position²

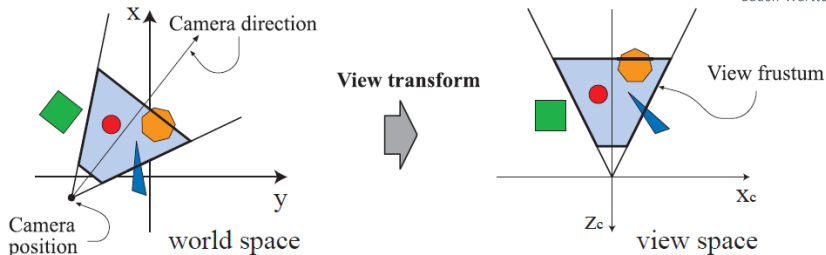


- ▶ Evaluate all other vertex-attributes³ such as:
 - Normals
 - Texture-Coordinates
 - Vertex-Color

²This can include blend and morph animation

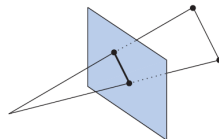
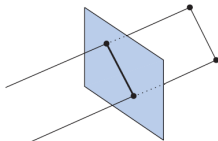
³These can be chosen freely by the programmer

Geometry Stage: Vertex Shading



```
default.vert  ▢ ×
1  #version 330 core
2  layout (location = 0) in vec3 aPos;
3
4  void main()
5  {
6      gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0);
7  }
```

Geometry Stage: Projection

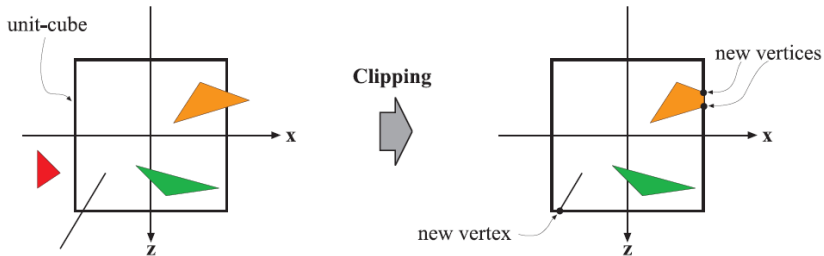


Orthographic Projection



Perspective Projection

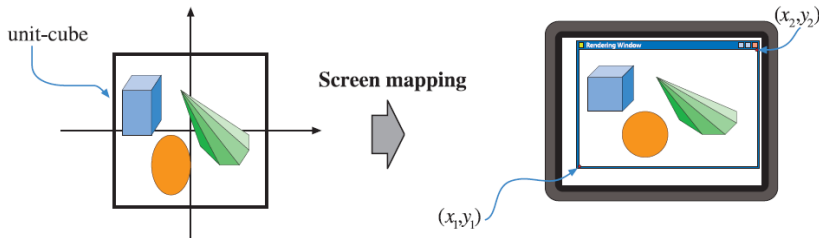
Geometry Stage: Clipping



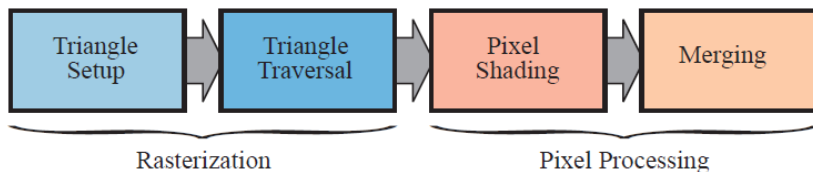
- ▶ Cull vertices which are outside of the unit cube
- ▶ At edges of cube new vertices are added to replace removed

⇒ Only continue with vertices which contribute to the image

Geometry Stage: Screen Mapping



Convert from *View Space* to *Screen Space* as last step of the *Geometry Processing Stage*

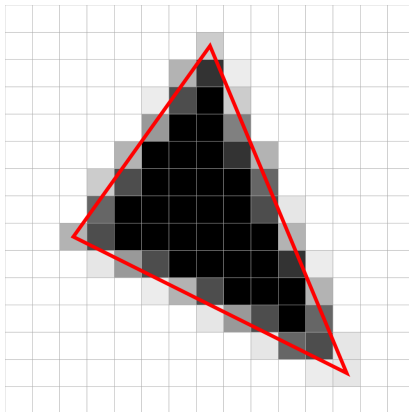


- ▶ Assembles all data from the Geometry Stage into batches of triangles for the following stages
- ▶ Uses fixed function⁴ hardware

⁴non programmable hardware functions

⁵Also called *Triangle Setup*

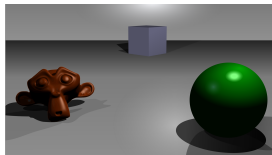
- ▶ Traversing through each triangle generating a *fragment* for each overlapping Pixel



- ▶ A Fragment Shader is applied to each fragment
- ▶ Uses interpolated data from the vertices of the triangle
- ▶ Passes a Color and Depth to the next stage



```
default.frag  ✕  
1  #version 330 core  
2  out vec4 FragColor;  
3  
4  void main()  
5  {  
6      FragColor = vec4(1.0f, 0.5f, 0.2f, 1.0f);  
7  }
```



A simple three-dimensional scene



Z-buffer representation

- ▶ Merge Pixel Shading result into the frame buffer based on depth
- ▶ Depth check can be made early to reduce performance cost for discarded fragments
- ▶ Each additional write to a pixel in the frame buffer is called overdraw
- ▶ Multiple Buffers may be used to pass data to postprocessing

- ▶ The whole pipeline can be parallelized
 - ▶ The execution order of the individual vertices, triangles and pixels can be chose arbitrarily
 - ▶ The whole pipeline can implement and executed on a CPU but there is alot of work to be done
- ⇒ Maybe parallel hardware is needed

How much Work needs to be done?

Lets assume we are not doing any overdraw. Then we will need still a lot of compute power for interactive framerates:

Resolution@FPS	Pixel/s ⁶	ms/frame
1080P@30	62.2 MP	33.3 ms
1080P@60	124.5 MP	16.6 ms
1080P@144	298.5 MP	6.9 ms
1080P@240	497.6 MP	4.2 ms
1440P@30	110.5 MP	33.3 ms
1440P@60	221.2 MP	16.6 ms
1440P@144	530.8 MP	6.9 ms
2160P@30	248.8 MP	33.3 ms
2160P@60	497.6 MP	16.6 ms

⇒ Specialized parallel hardware is needed

⁶In million pixels per second

CPU	vs	GPU
4-32	Cores	500-15000+
4-8	SIMD	8-64
5-6 GHz	Frequency	2 GHz
branch prediction	latency-	quick
cache prefetching	strategy	context switch

Modern GPU can handle shader Invocations with over a million threads.

GPU cores refers to CUDA Cores (which are not the completely comparable to CPU cores).

Lets say we need to execute 2000 fragments for one triangle:

- ▶ The GPU bundles the Invocations into groups called *warps* by NVIDIA and *wavefronts* by AMD
- ▶ With a 32 Thread SIMD core we need 62.5 wavefronts for the shader
- ▶ The shader is executed 63 times with a width of 32
- ▶ When a fetch is encountered a new wavefront is started on the core unit the fetch is completed

The occupancy of a GPU is measured by the number of wavefronts *in flight* on the GPU.

The GPU - Hardware

For a deeper dive into modern GPU Hardware you can take a look at this Presentation from 2020 by AMD:



Any Questions?

